ADR_014 模組內採傳統分層並集中跨模組呼叫

> 狀態：已接受
> 日期：2026-10-15

背景

ADR_001 決定以模組化單體起步，《設計_003_系統架構》第 5 節規定每個模組分為「對外公開的頂層套件」與「`internal` 子套件」，但沒有規定 `internal` 內部的結構。

第一個模組（帳號）完成時，`internal` 內的類別全部平放。需要考慮的情況：

- 訂單、付款等核心模組會比帳號模組大得多，平放不易閱讀
- 開發者熟悉的是傳統分層（controller、service、repository、entity）
- 之後預計拆成微服務（設計_003 第 7 節，第 8 階段），模組之間的同步呼叫要改成網路呼叫

考慮過的選項

選項	優點	缺點
模組內平放	最簡單	模組變大後不易找到程式；沒有規則約束誰能依賴誰
整個專案依層分（`controller/`、`service/` 在最上層）	最常見的傳統分層	Spring Modulith 會把各層當成模組，模組邊界消失，等於推翻 ADR_001
先依模組切，模組內採傳統分層	保留模組邊界；模組內的結構符合熟悉的習慣	跨子套件使用的類別要改為 `public`（仍只在模組內）
先依模組切，模組內採六角形架構（port 與 adapter，領域層不依賴框架）	領域邏輯最乾淨，替換外部技術最容易	介面與轉換程式多，以本專案的規模成本偏高

模組內的分層對拆服務幫助不大：拆分時要處理的是模組邊界、各自的 schema、模組之間的呼叫與事件，這些都由「依模組切」決定。真正影響拆分成本的，是呼叫其他模組的程式散落在哪裡。

決定

先依模組切，模組內採傳統分層；另外把與其他模組互動的程式集中在 `client` 子套件。這是六角形架構中「port 與 adapter」概念的簡化版，只用在跨模組的地方。

子套件	內容
`controller`	實作 OpenAPI 產生的介面；請求與回應的轉換
`service`	業務流程與交易邊界

子套件	內容
`repository`	資料存取：Spring Data 的 repository 與直接寫 SQL 的類別
`entity`	JPA 實體與其列舉
`client`	與其他模組互動：呼叫其他模組公開的介面、訂閱其他模組的事件
`config`	模組自己的設定與 Spring 設定類別

規則由架構測試檢查：

- 依賴方向：`controller` → `service` → `repository`、`entity`、`client`；下層不依賴上層。例外是 `client` 中訂閱事件的程式，收到事件後呼叫 `service` 處理
- `internal` 內的類別都必須放在上述子套件之一
- 只有 `client` 可以使用其他業務模組的型別；`shared` 與 OpenAPI 產生的程式不在此限
- OpenAPI 產生的型別只在 `controller` 使用，`service` 不依賴 API 的資料格式

`shared` 不是業務模組，其 `internal` 依功能分子套件（例如 `security`、`audit`），不套用上述分層。

後果

- 好處：模組邊界與 ADR_001 不變；模組內的結構一眼可見；日後拆出某個模組時，只需把呼叫它的各模組的 `client` 改為網路呼叫或訊息佇列，`service` 不需修改
- 代價：跨子套件使用的類別須為 `public`，只能靠 Spring Modulith 的驗證測試防止其他模組使用；每次跨模組呼叫都多一層 `client`
- 重新檢討的條件：若某個模組的業務規則複雜到需要與框架完全隔離，該模組改採六角形架構